AI Driver Assistant using deep learning

Supervised by

Dr. Maged Askar

Faculty of Computers and Information
FCI

2/ 7 / 2025

Structure of the Presentation

1. Introduction
2. Problem Statement

Structure of the Presentation (Cont.)

3 Proposed Solution

- Facial Landmark-Based Approach
  - Process
  - Technologies and Frameworks Used
  - Visualization
- YOLO-Based Deep Learning Approach
  - Process
  - Technologies and Frameworks Used
  - Visualization
- Comparison of Methods
- YOLO-Based Deep Learning Approach
  - Process
  - Technologies and Frameworks Used
  - Visualization
- YOLO-Based Deep Learning Approach
  - Process
  - Technologies and Frameworks Used
  - Visualization

Structure of the Presentation (Cont.)

4. System Architecture
5. Implementation
6. Challenges and Limitations
7. Results and Evaluation
8. Future Work
9. Conclusion

Introduction

Road traffic accidents remain a major global challenge, with approximately **1.19** million fatalities each year. Beyond the loss of life, these accidents impose severe financial burdens, costing nations an estimated **3%** of their Gross Domestic Product (GDP) annually. Many of these accidents result from human factors, such as driver fatigue, lack of awareness, and failure to detect hazards in time.

Introduction

While advancements in road design and infrastructure—such as footpaths, cycling lanes, and traffic-calming measures—have helped reduce risks, road traffic accidents remain a major concern. Alarmingly, drowsy driving alone has caused nearly **30,000** fatalities between 2017 and 2021, with drivers experiencing impairment equivalent to a blood alcohol content (BAC) of **0.10%** after 24 hours of wakefulness.

Introduction

To address these challenges, our project introduces an AI Driver Assistant system to enhance road safety and reduce accidents, and overall driver safety by providing real-time alerts.

Problem Statement

Despite progress in road safety measures, driver fatigue, poor road conditions, and low-visibility hazards remain significant contributors to road accidents. For example:

- Drowsy driving impairs a driver's ability to focus and react, leading to preventable accidents.
- Small or distant objects on the road, such as pedestrians, animals, or stones, are often missed due to driver inattention.
- Road hazards like potholes, bumps, and obstacles pose serious risks, especially in poorly maintained areas.

Problem Statement

Our project aims to address these challenges by developing three detection models:

- A Drowsiness Detection Model to monitor driver fatigue in real-time.
- A Road Object Detection Model to identify small or distant objects found in the road
- A Hazard Detection Model to detect road hazards like potholes, obstacles, and bumps.

Problem Statement

By integrating these models, we aim to build an integrated and effective safety system. This system will help drivers stay more alert, minimize the chances of accidents, and most importantly, save lives and keep people safe. Our goal is to make driving safer for everyone through the use of smart technology that works to reduce harm and save lives.

Proposed Solution

Our system implements two approaches for detecting driver drowsiness:

1. **Facial Landmark-Based Approach**
2. **YOLOv8-Based Deep Learning Approach**

And one approache for object detection model and road hazards detection model:

1. **YOLOv10-Based Deep Learning Approach**

Proposed Solution

This method relies on the `face_recognition` library and facial landmarks to identify drowsiness indicators like eye closure and mouth opening.

Proposed Solution

- The system reads an input video frame and converts it to RGB using OpenCV.
- Face landmarks (eyes and mouth) are detected using the `face_recognition` library.
- The **Eye Aspect Ratio (EAR)** is calculated to determine if the eyes are closed. A predefined threshold of **0.20** is used.
- The **Mouth Aspect Ratio (MAR)** is computed to detect yawning, using a threshold of **0.24**.
- If a driver shows drowsiness signs for more than **5 consecutive frames**, an audio alert is triggered using the `pygame` library.

Proposed Solution

- **face_recognition** – Facial landmark detection.
- **OpenCV** – Image and video processing.
- **scipy** – Euclidean distance calculations for EAR and MAR.
- **pygame** – Audio alert generation.

Proposed Solution

- The system overlays detected facial landmarks on the input frame.
- When drowsiness is detected, the word **"Drowsy"** is displayed in red on the screen.

Proposed Solution

This method leverages a custom-trained **YOLOv8** model to detect drowsiness-related objects in real-time.

Proposed Solution

- The **YOLOv8** model is loaded and used to process video frames.
- The model detects **closed eyes (class 0)** and **open mouth (class 1)** using bounding boxes and confidence scores.
- If a detection confidence score exceeds a predefined threshold (**0.2 for eyes, 0.5 for mouth**), an **audio alert** is triggered using the `playsound` library.
- The system provides real-time feedback by displaying annotated frames with detected objects.

Proposed Solution

- **ultralytics** – YOLO model implementation.
- **OpenCV** – Video stream processing.
- **playsound** – Alert sound playback.

Proposed Solution

- The system overlays bounding boxes on detected facial features.
- Drowsiness alerts are triggered in real-time when conditions are met.

Proposed Solution

Approach	Advantages	Limitations
Facial Landmark-Based	Simple and efficient, works in real-time	Sensitive to lighting and head orientation
YOLO-Based	Robust and accurate, better in low-light conditions	Requires high computational power and dataset training

Table: Comparison of Facial Landmark-Based and YOLO-Based Approaches

Proposed Solution

This method leverages a custom-trained **YOLOv10** model to detect objects detection model in real-time.

Proposed Solution

- The **YOLOv10** model is loaded and used to process video frames.
- The model detects **Objects** like (**People, Cats, Dogs, Cars,Traffic sign, Motorcycles, Vans and so on**) using bounding boxes and confidence scores.
- The system provides real-time feedback by displaying annotated frames with detected objects.

Proposed Solution

- **ultralytics** – YOLO model implementation.
- **OpenCV** – Video stream processing.
- **playsound** – Alert sound playback.

Proposed Solution

- The system overlays bounding boxes on detected objects features.
- Objects alerts are triggered in real-time when conditions are met.

Proposed Solution

Proposed Solution

This method leverages a custom-trained **YOLOv10** model to detect road hazards detection model in real-time.

Proposed Solution

- The **YOLOv10** model is loaded and used to process video frames.
- The model detects **Road hazards** like (**Speed bumps, Potholes, Cracks, Manholes and so on**) using bounding boxes and confidence scores.
- The system provides real-time feedback by displaying annotated frames with detected objects.

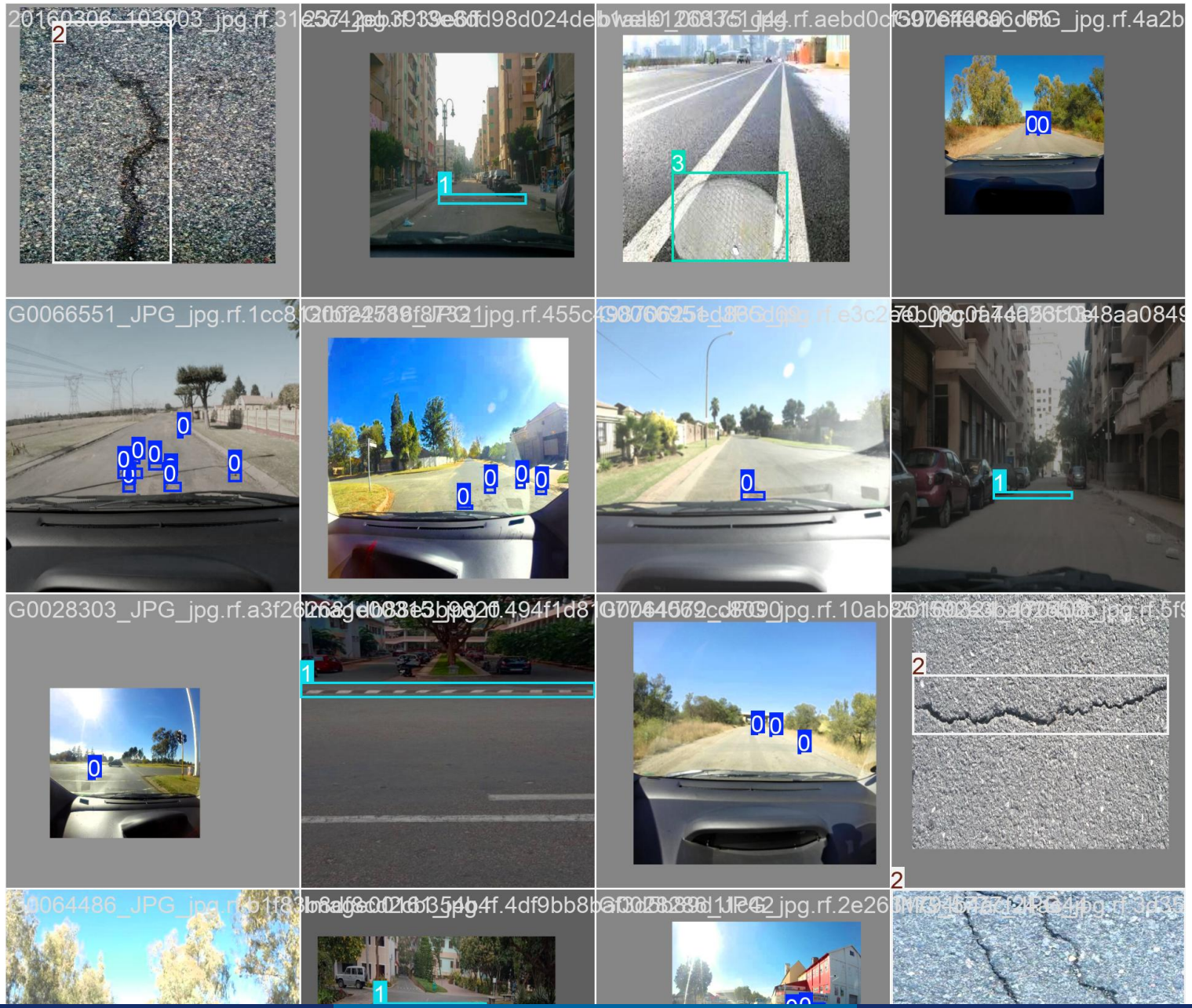
Proposed Solution

- **ultralytics** – YOLO model implementation.
- **OpenCV** – Video stream processing.
- **playsound** – Alert sound playback.

Proposed Solution

- The system overlays bounding boxes on detected road hazards features.
- Road hazards alerts are triggered in real-time when conditions are met.

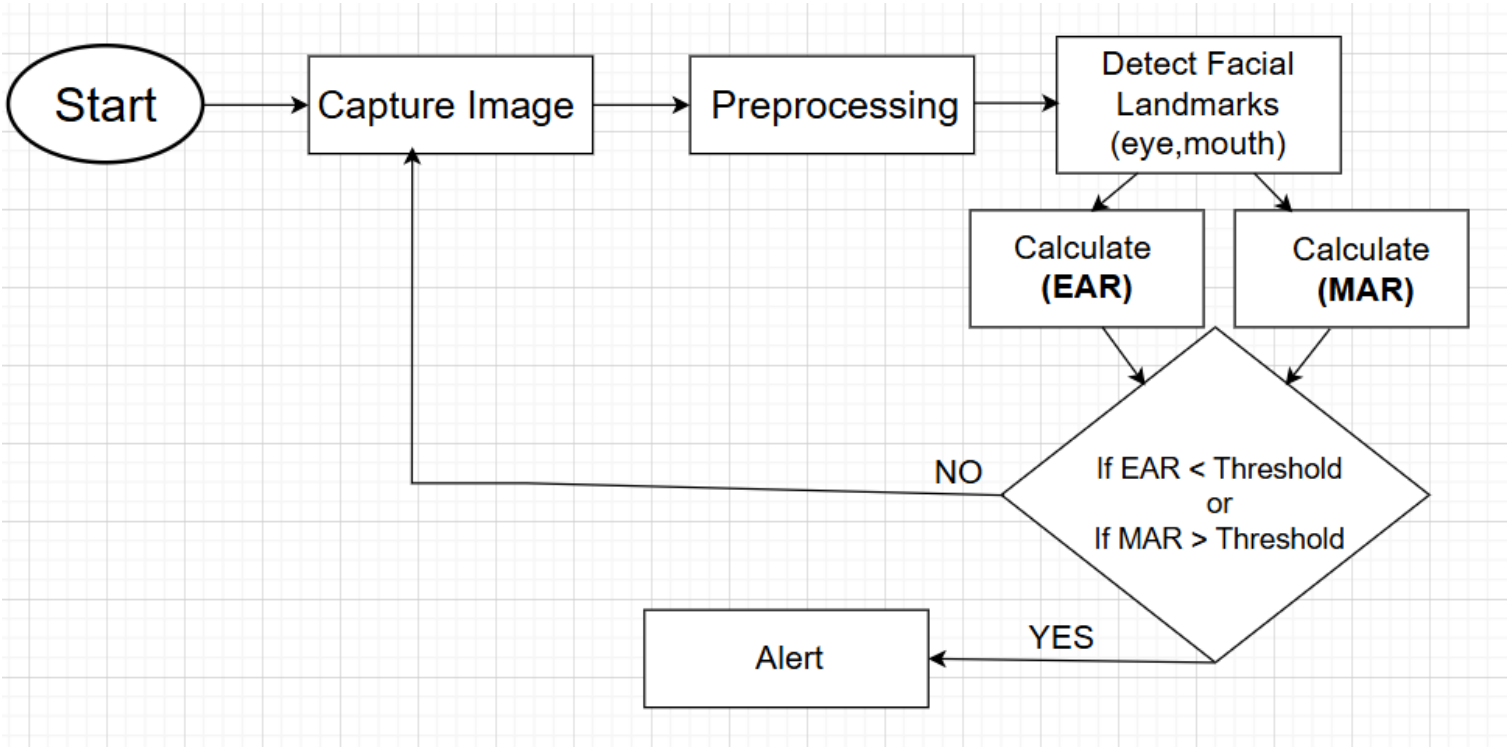
Proposed Solution



System Architecture

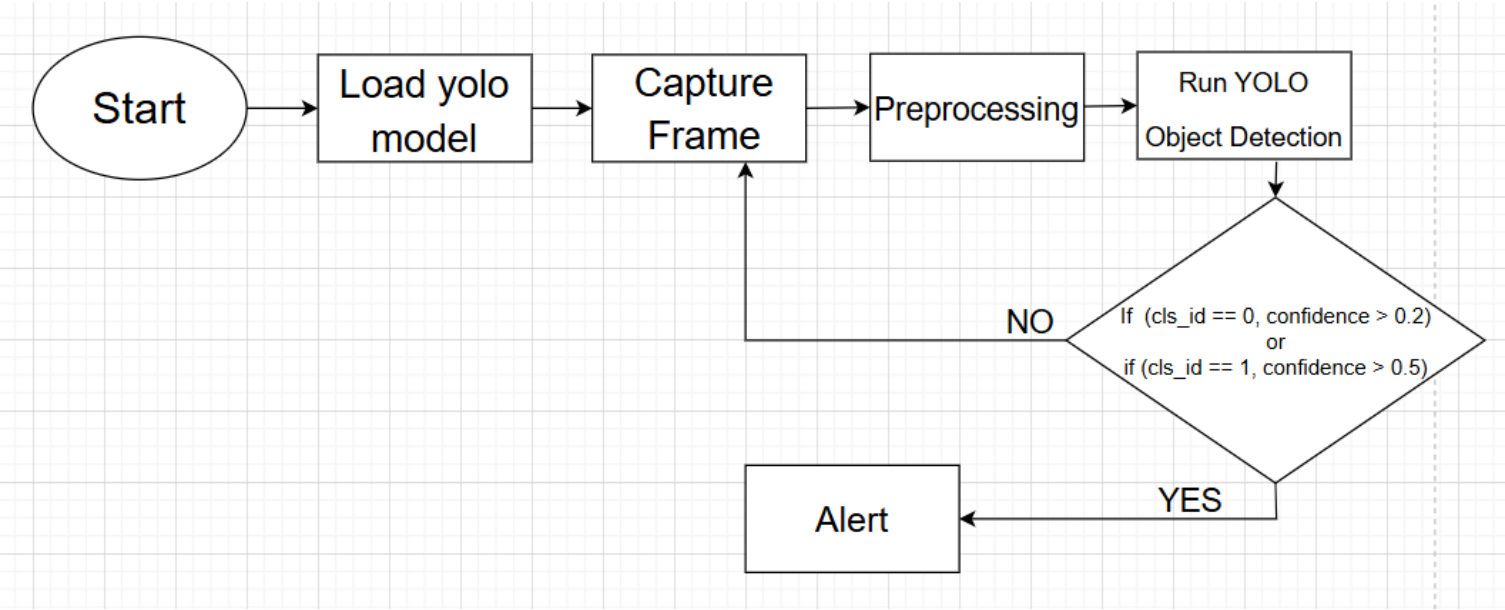
The system includes two Drowsiness Detection Models, each using different approaches for preprocessing and detection:

1. Facial Landmark-Based Model



System Architecture

1. YOLO-Based Mode



Implementation

Facial Landmark Model:

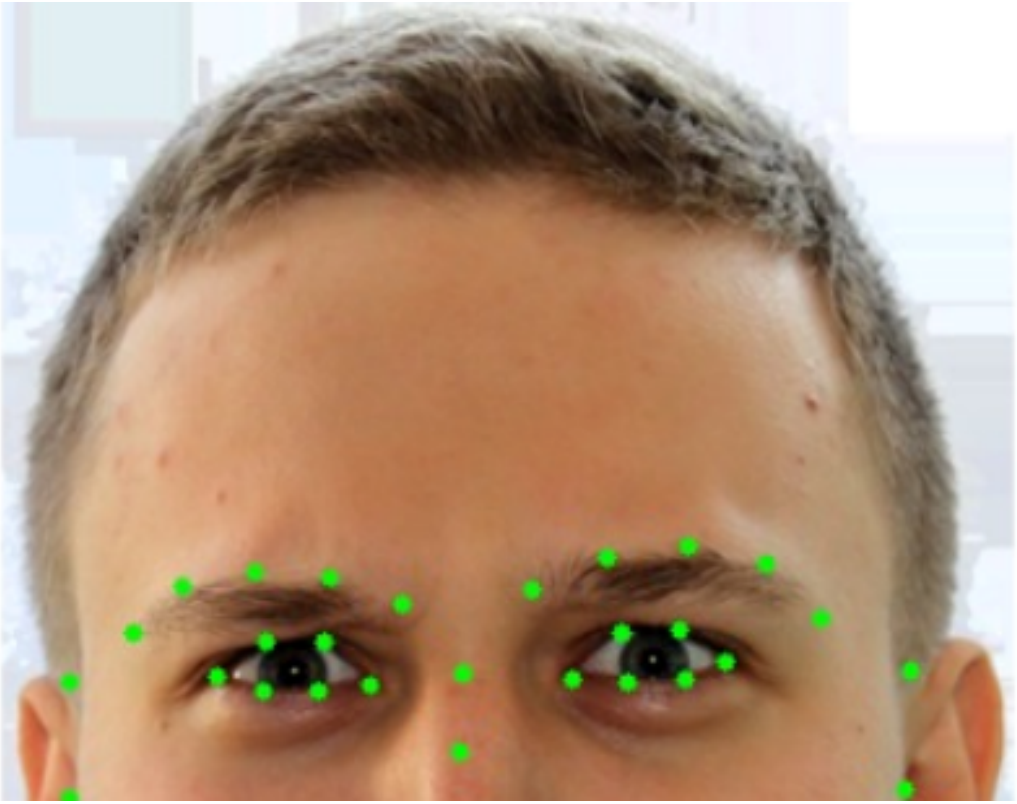
Computes **Eye Aspect Ratio (EAR)** and **Mouth Aspect Ratio (MAR)**.

- EAR

  The formula used for calculating EAR is:

$$EAR = \frac{\|P_2 - P_6\| + \|P_3 - P_5\|}{2 \times \|P_1 - P_4\|} \tag{1}$$

Implementation



- A predefined threshold of **0.20** is used.

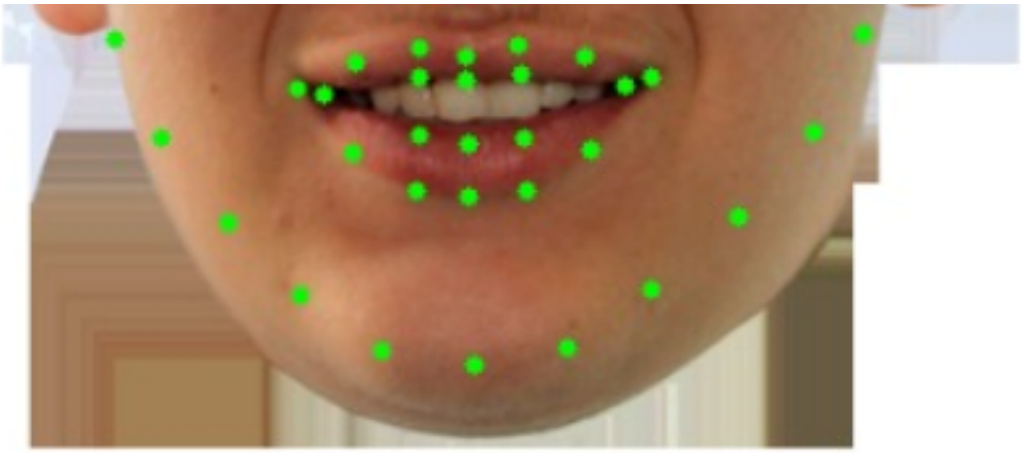
Implementation

- MAR

  The formula used for calculating is:

$$MAR = \frac{\|P_2 - P_4\| + \|P_3 - P_5\|}{2 \times \|P_1 - P_6\|} \tag{2}$$

Implementation



- A predefined threshold of **0.24** is used.

Implementation

YOLOV8-Base Drowsiness Detection:

- Dataset size:
  - Training Set: 1400 images
  - Validation Set: 400 images
  - Test Set: 200 images
- Dataset classes:
  - Drowsy
  - Yawning
- Training Dataset:
  - Batch Size: 16
  - Epochs: 500

Implementation

- **Technologies Used**
  - **ultralytics** for YOLO model implementation.
  - **pygame** for audio alert generation.
  - **OpenCV** for video stream processing.
  - **pyttsx3** for text-to-speech voice alerts to the drive
  - **python-telegram-bot** for receiving GPS coordinates via Telegram location sharing.
  - **Twilio** for sending SMS messages with the driver's location when critical fatigue is detected.

Implementation

Road Object Detection:

- Dataset size:
  - Training Set: 15470 images
  - Validation Set: 4420 images
  - Test Set: 2210 images
- Dataset classes:
  - People
  - Animals (like Cats, Dogs)
  - Traffic sign
  - Vehicles (like Cars, and Motorcycles)
- Training Dataset:
  - Batch Size: 64
  - Epochs: 300
  - workers: 4

Implementation

- **Technologies Used**
  - **YOLOv10** Object detection model used to identify road hazards in real-time.
  - **OpenCV** for video stream processing.
  - **pyttsx3** Converts text-based alerts to speech.
  - **pyodbc** Communicates with SQL Server for logging detected locations.
  - **NumPy –** Assists in spatial calculations like object center and distance estimation.
  - **EasyOCR** - extract text from images
  - SQL Database Logging: Logs each hazard's location to a SQL Server database.
  - **Telegram Bot API –** Handles location updates and automates data transmission.

Implementation

Road Harazard Detection

- Dataset size:
  - Training Set: 4758 images
  - Validation Set: 1359 images
  - Test Set: 679 images
- Dataset classes:
  - Speed bumps
  - Potholes
  - Cracks
  - manhole
- Training Dataset:
  - Batch Size: 64
  - Epochs: 200
  - workers 4

Implementation

- **Technologies Used**
  - **YOLOv10** Object detection model used to identify road hazards in real-time.
  - **OpenCV** for video stream processing.
  - **pyttsx3** Converts text-based alerts to speech.
  - **pyodbc** Communicates with SQL Server for logging detected locations.
  - **NumPy** Assists in spatial calculations like object center and distance estimation.
  - **SQL Database Logging** Logs each hazard's location to a SQL Server database.
  - **Telegram Bot API** Handles location updates and automates data transmission.

Challenges and Limitations

1. **Data Annotation**
   - ★ Roboflow

2. **second model**
   - ★ Collecting Dataset
   - ★ Training
     - •Slow
     - •Heaviness - RTX 3070 ti

3. **Getting Location**
   Two approaches:
   - ★ WebDriver-Based GPS with Fallback to IP Geolocation **Confusion Matrix:**

84	27.181	31.1837	10000	شارع 26 يوليو، 47، أسيوط، أسيوط، مصر	أسيوط	ip	2025-07-02 07:58:2

Challenges and Limitations

- Telegram Bot-Based Location Collection

id	latitude	longitude	accuracy	address_ar	governorate_ar	source	timestamp
83	27.187927	31.189087	18	شارع الوحدة، سابق، أسيوط، مصر	أسيوط	NULL	2025-07-02 07:57:09.630

Results and Evaluation

Facial Landmark Model:

- Model Performance

  The performance of the landmarks-based drowsiness detection model is summarized as follows:

  Model Accuracy: 68%

Results and Evaluation

Precision and Recall per Class:

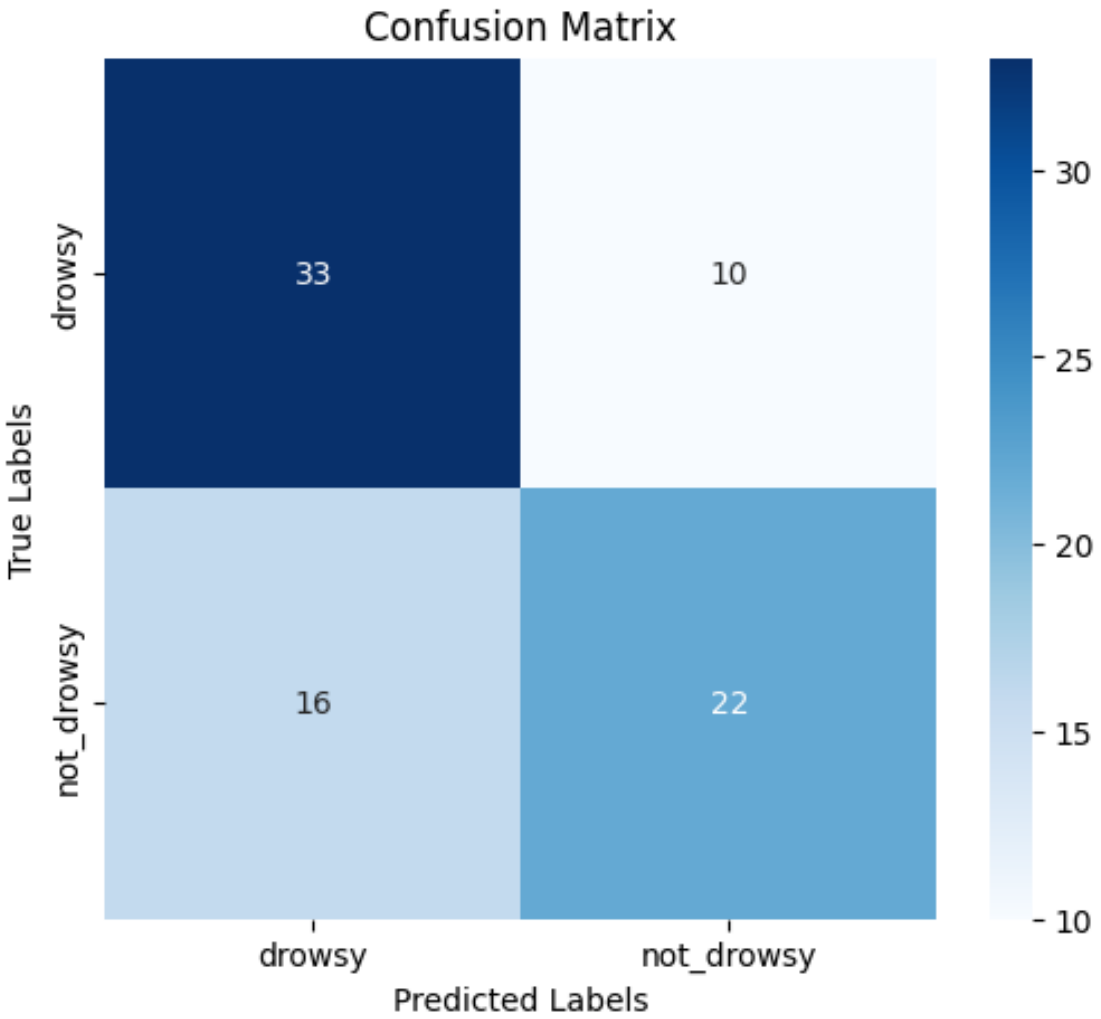
Class	Precision	Recall
Drowsy	0.67	0.77
Not Drowsy	0.69	0.58

Additional Metrics:

Metric	Precision	Recall
Macro Average	0.68	0.67
Weighted Average	0.68	0.68

Results and Evaluation

Confusion Matrix:



Results and Evaluation

YOLOv8-based drowsiness detection model:

- Model Performance

  The performance of the YOLOv8-based drowsiness detection model was assessed using standard object detection metrics. Below table summarizes the evaluation results on the test dataset.

Results and Evaluation

Metric	Value
Precision	0.96
Recall	0.98
F1-score	0.97
mAP@50	0.96
mAP@50-95	0.68

Table: Performance Metrics of the Drowsiness Detection Model.

Results and Evaluation

It makes very few false detections (high precision).

It rarely misses actual objects (high recall).

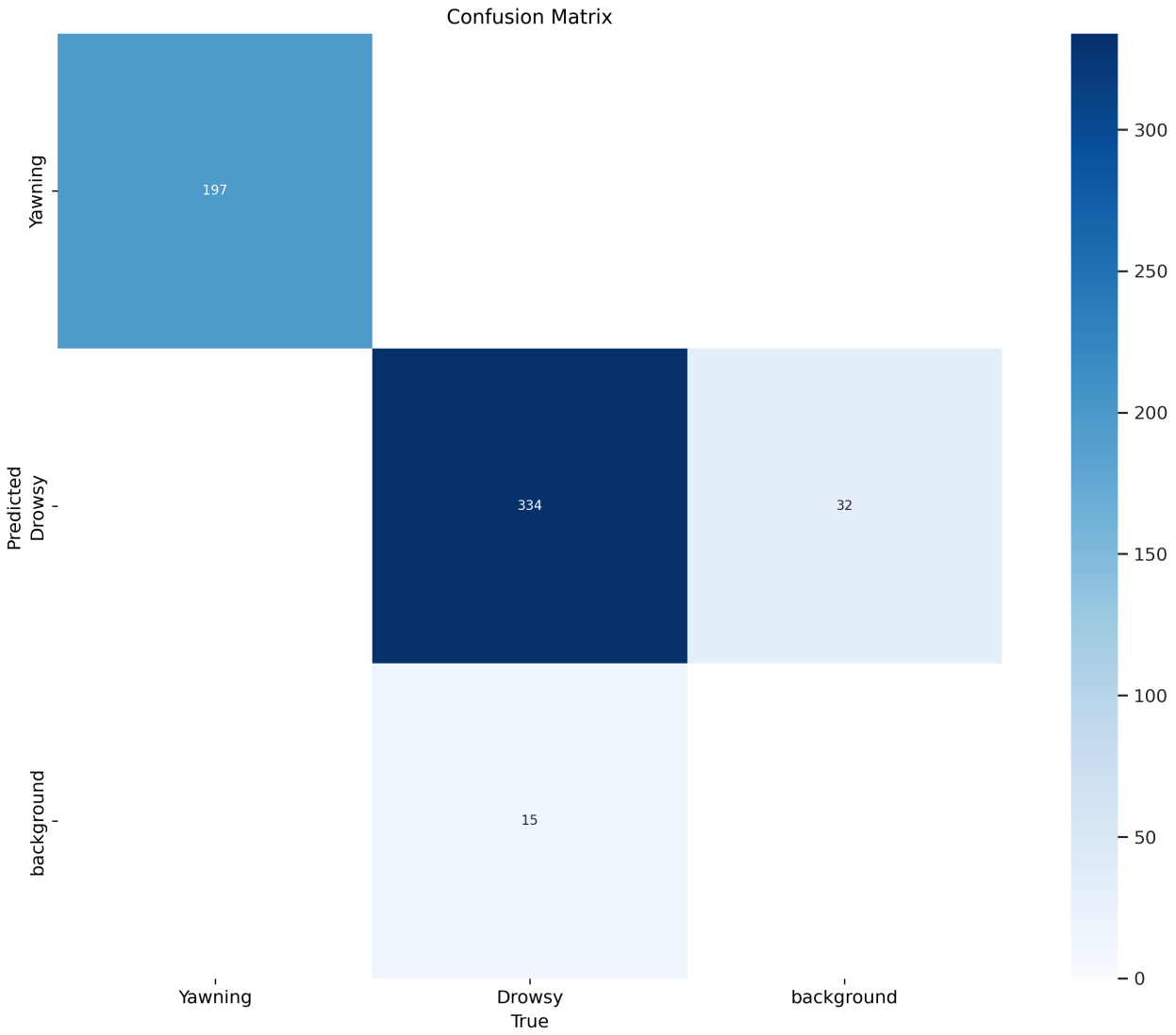
F1-Score : the model has an excellent balance between precision (accuracy of detections) and recall (ability to detect all actual cases).

A high mAP@50 (0.96) means that the model is excellent at detecting objects when lenient IoU thresholds are applied.

A moderate mAP@50-95 (0.68) indicates that as stricter IoU thresholds are used, the model's performance drops, but it is still fairly reliable.

Results and Evaluation

Confusion Matrix:



Results and Evaluation

Road Object Detection Model:

- Model Performance

  The performance of the YOLOv10-based drowsiness detection model was assessed using standard object detection metrics. Below table summarizes the evaluation results on the test dataset.

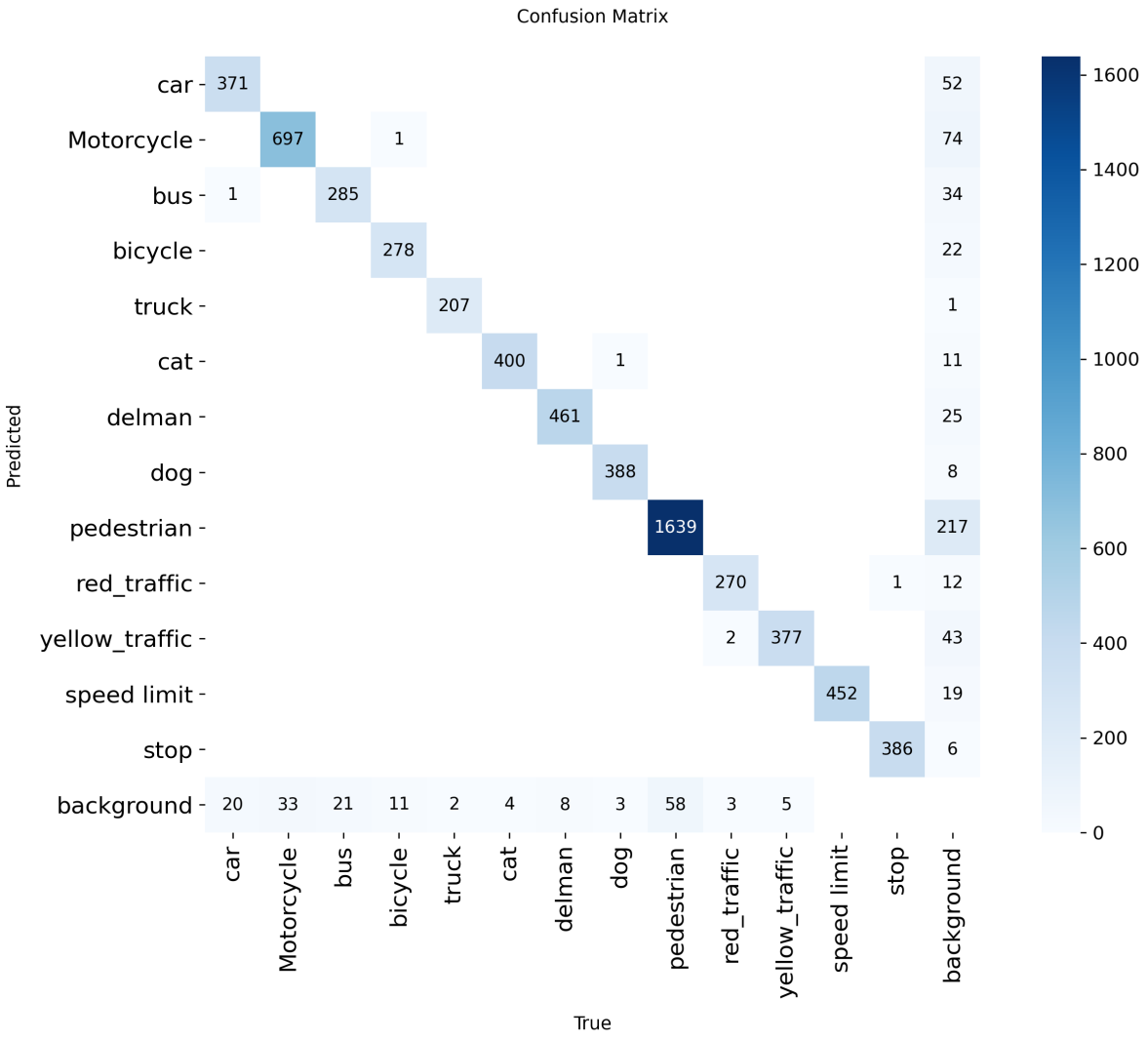
Results and Evaluation

Metric	Value
Precision	0.75
Recall	0.75
F1-score	0.75
mAP@50	0.98
mAP@50-95	0.86
mAP@75	0.94

Table: Performance Metrics of the Object Detection Model.

Results and Evaluation

Confusion Matrix:



Results and Evaluation

Road Hazards Detection Model:

- Model Performance

  The performance of the YOLOv10-based road hazards detection model was assessed using standard road hazards detection metrics. Below table summarizes the evaluation results on the test dataset.

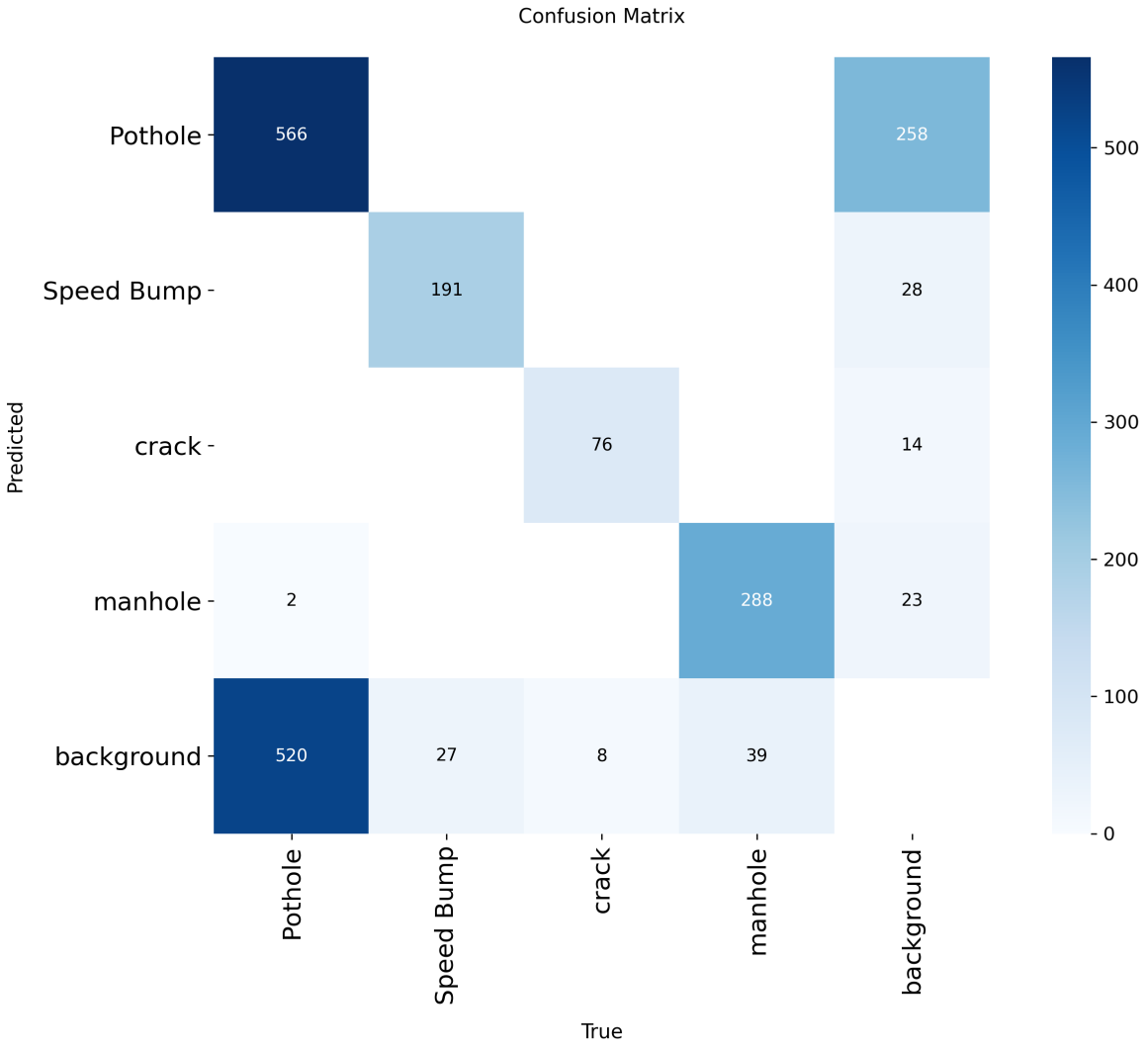
Results and Evaluation

Metric	Value
Precision	0.75
Recall	0.75
F1-score	0.75
mAP@50	0.81
mAP@50-95	0.54
mAP@75	0.56

Table: Performance Metrics of the Road hazards Detection Model.

Results and Evaluation

Confusion Matrix:



Future Work

To overcome the current performance limitations and bring the system closer to production-level deployment, our future work will focus on enhancing both the efficiency and reliability of the models used

1. Model Optimization
   Use quantization and pruning to make YOLOv8 models smaller and faster without losing accuracy.

2. Faster Execution
   Convert models to ONNX or TensorRT to speed up inference and reduce power use.

Future Work

1. Edge Deployment
   Run the system on edge devices like NVIDIA Jetson or Google Coral for real-time use in cars.

2. Better Performance
   Use smart techniques like skipping frames and running processes in parallel to keep the system fast and efficient.

These upgrades will make the system more reliable, responsive, and ready for deployment in smart vehicles.

Conclusion

This project shows how deep learning can help build a real-time driver assistance system. It combines three YOLO models to detect drowsiness, road signs, speed limits, and road hazards, all inside one simulation program with live video and voice alerts. While the system works well, using all

three models at once on a regular laptop caused slow performance. To fix this, we plan to improve the models and run them on faster AI devices.

Conclusion

In the future, this system can be used in smart cars to give real-time safety alerts and keep drivers more aware. Adding GPS and sensors can make it even better. Overall, the project

proves that deep learning can make driving safer and smarter.

Thanks!